

INFORMAÇÕES		
MÓDULO	AULA	INSTRUTOR
07	ASP.NET Core Web API – Avançado - Padrões de Projeto, Arquitetura Limpa e Alta Performance	08 - Arquitetura de Alta Performance e CQRS Pragmático – Revisão Final
		Guilherme Paracatu

1. Revisão Final do Módulo

1. A Divisão de Responsabilidades (CQRS)

- **Queries (Leituras):** Otimizadas exclusivamente para a tela do usuário. Não trafegamos Entidades de Domínio puras; utilizamos **DTOs Achatados (Flat)** para entregar exatamente o que o *Front-end* precisa.
- **Commands (Escritas):** Orientados ao comportamento e às regras de negócio. São os responsáveis por alterar o estado da aplicação com segurança.
- **O Pragmatismo:** Embora a teoria purista diga que um *Command* deve retornar void, no mundo real das APIs Web nós adaptamos a regra. Retornamos um pacote padronizado (CommandResult) contendo o ID gerado, poupando o *Front-end* de fazer novas requisições e eliminando o *overhead* de rede.

2. Otimização Extrema com Entity Framework Core 8

- **Projeções (.Select):** Esta é a nossa regra de ouro da performance. Em vez de carregar todas as colunas e relações para a memória usando o `.Include()`, usamos o `.Select()` para pedir ao banco de dados *apenas* os dados estritamente necessários para o DTO. Isso reduz drasticamente o tráfego de rede e o consumo de RAM.
- **Trade-offs de Performance (LINQ vs. SQL Puro):** O LINQ resolve 95% dos problemas do dia a dia, garantindo segurança de tipagem e refatoração automática. O SQL Puro (SqlQuery) é a nossa "ferramenta de precisão", acionada apenas em gargalos de performance comprovados pelo *Profiler* ou quando precisamos de recursos nativos do SGBD (como CTEs recursivas, PIVOT ou HINTS).

3. O Padrão Mediator e o Mensageiro Inteligente (Message Bus)

- **Desacoplamento e SRP:** A nossa Controller não recebe mais múltiplos Repositórios via injeção de dependência. Ela recebe apenas o IMessageBus (nosso Mediator). Isso garante o Princípio da Responsabilidade Única (SRP), deixando a Controller focada apenas em gerenciar a requisição HTTP. Toda a complexidade de negócio fica isolada no *Handler*.
- **Pipeline de Execução:** O nosso InMemoryBus atua como um maestro. Ele registra e executa os *Handlers*, e possui a inteligência de garantir que a resposta principal (aquela que contém dados reais, como o ID gerado no banco) **não seja sobrescrita** por resultados neutros gerados por *Handlers* secundários (como rotinas de Log ou Auditoria).

4. A Consistência com Unit of Work (UoW)

- **Atomicidade (Transações):** Removemos o SaveChanges() de dentro do Repositório para evitar o perigo das transações parciais (ex: conseguir salvar uma *Marca*, mas dar erro no *Veículo* logo em seguida, deixando o banco inconsistente).
- **O Ponto de Confirmação:** Na nossa arquitetura, o *Handler* orquestra tudo em memória usando os Repositórios. Na última etapa do processo, quando temos certeza de que todas as regras de negócio passaram, invocamos o `_uow.CommitAsync()`. Assim, garantimos que ou tudo é salvo perfeitamente de uma só vez, ou nada é salvo (*Rollback* automático).

5. O Domínio do Assincronismo no C#

- **A Regra do Void:** Nunca utilize `async void` em métodos da aplicação. Isso causa falhas silenciosas que não podem ser capturadas e acabam derrubando o servidor.
- **Otimização de Tasks:** Se um método precisa retornar uma Task (devido a um contrato de interface), mas não possui nenhum processamento assíncrono interno (nenhum `await`), **não coloque a palavra `async` na assinatura**. Isso evita o desperdício de memória com a Máquina de Estados invisível e elimina o aviso CS1998. Para cumprir o contrato de forma limpa, utilize `Task.FromResult()` ou `Task.CompletedTask`.